Code Review and Refactoring: Linear Probing of Belief-State Geometry in Transformer Residual Streams

C. L.

March 6, 2026

Contents

1 Purpose of This Document

This report is a code review of `old_src/pca.py`, which implements the pipeline for extracting fractal belief-state geometry from transformer residual streams. The review separates the *conceptual* content (what is being computed and why) from *engineering* concerns (how the code is structured, its scalability, and its correctness).

2 Conceptual Background

2.1 Belief States as an Iterated Function System

A Hidden Markov Model with N hidden states and vocabulary V defines a Bayesian belief update rule. After observing token x_t, the belief vector $\alpha_t \in \Delta^{N-1}$ (the probability simplex) evolves as

$$\alpha_{t+1} \;=\; \frac{E[x_t]^\top \alpha_t}{\mathbf{1}^\top E[x_t]^\top \alpha_t}, \tag{1}$$

where $E[x] \in \mathbb{R}^{N\times N}$ is the emission matrix for token x, with $E[x]_{ik} = P(\text{observe } x \text{ and transition } i \to k)$. Each token defines a contraction on the simplex; together the $|V|$ maps $\{f_x : x \in V\}$ form an **Iterated Function System** (IFS). The attractor of this IFS—the closure of all reachable belief states—is the **mixed-state presentation** (MSP). For generic HMMs this attractor is a fractal subset of the simplex [3, 4].

2.2 The Probing Question

Shai et al. [1] showed that transformers trained on next-token prediction over HMM-generated data learn an internal representation of the belief state that is recoverable via a *linear* map from the residual stream. Concretely, if $h_t^{(\ell)} \in \mathbb{R}^d$ denotes the residual-stream activation at layer ℓ and position t, then there exists a matrix $W \in \mathbb{R}^{N\times d}$ and bias $b \in \mathbb{R}^N$ such that

$$W h_t^{(\ell)} + b \;\approx\; \alpha_t. \tag{2}$$

The quality of this approximation, measured by R^2, quantifies how faithfully the transformer encodes the belief state at each layer.

2.3 Why "Fractal Directions"?

Because the belief simplex is $(N-1)$-dimensional (2D for Mess3's 3 states), only a low-dimensional subspace of the d-dimensional residual stream is needed to encode α_t. The principal components of this subspace are called the **fractal directions**: projecting activations onto them recovers the fractal geometry of the MSP. Ablating these directions destroys predictive performance far more than ablating the orthogonal complement (Section 5 of the time-reversal report), confirming their causal role.

3 What the Code Computes

The pipeline in `old_src/pca.py` has four stages:

1. **Model loading** (`load_model_from_weights`): Deserialise a trained transformer (either `HookedTransformerM` via TransformerLens or `MaskedHeadTransformerModel` via NNsight) from a checkpoint directory containing `config.yaml` and a `.pt` weights file.

2. **Activation extraction** (`extract_residual_stream`): Run a forward pass on a batch of observation sequences and record the post-layer residual-stream activations. Returns a tensor of shape `(num_layers, batch, seq_len, d_model)`.

3. **Linear probing** (`learn_affine_mapping`, `pca_layer_wise_actvs`, `pca_concat_actvs`): Fit ordinary least-squares regression from activations to ground-truth belief states. Two modes:

   - *Layer-wise*: One regression per layer, answering "at which layer does the belief representation crystallise?"
   - *Concatenated*: Stack all layers into a single feature vector, answering "is belief information distributed across layers?" (relevant for RRXOR, where a single layer's R^2 can be as low as 0.31).

4. **Visualisation** (`barycentric_to_cartesian_2d`, `plot_in_simplex`): Project 3D barycentric coordinates to an equilateral triangle for plotting.

The entry point `run_pca_and_save_results` orchestrates all four stages over one or more model directories, with optional MPI parallelism.

4 Conceptual Critique

4.1 The Name "PCA" Is a Misnomer

Despite the filename, **no PCA is performed on the activations**. The code fits a linear regression (ordinary least squares via `sklearn.LinearRegression`) from activations to belief states. PCA and linear regression are related—PCA finds the directions of maximum variance in the *input* space, while regression finds the directions that maximally predict a *target*—but they are distinct operations. The code should be renamed (e.g., `linear_probe.py` or `belief_probe.py`) to avoid confusion.

The `sklearn.decomposition.PCA` import on line 12 is in fact *unused*.

4.2 Appropriateness of a Linear Probe

The choice of a linear (affine) probe is well motivated: it tests whether the belief state is linearly decodable from the residual stream. A nonlinear probe (e.g., an MLP) would achieve higher R^2 but would not distinguish "the information is linearly accessible" from "the information exists somewhere in the activations and can be extracted with sufficient computation." The linear probe is the standard tool in the mechanistic-interpretability literature for this reason [1, 5].

One subtlety: the probe includes a bias term (b in the affine map), which is important because belief states are constrained to the simplex ($\sum_i \alpha_i = 1$) while activations are not. The bias absorbs the mean of the belief distribution.

4.3 Last-Position vs. All-Position Analysis

The `use_last_pos` flag (default `True`) restricts the analysis to the final sequence position. This is the natural choice for autoregressive models: the last-position activation has seen the full context

and should encode the most refined belief state. When `use_last_pos=False`, the code reshapes all positions into a single regression, mixing different context lengths. This is a weaker test—short-context positions will have diffuse beliefs (close to the stationary distribution) and inflate R^2 by providing "easy" data points near the simplex centre.

4.4 Metric Interpretation

Three metrics are reported:

- **MSE**: Mean squared error between predicted and true belief states, averaged over all simplex dimensions and data points.
- R^2: Coefficient of determination. Values above 0.99 indicate near-perfect linear decodability.
- **Baseline MSE**: The MSE achieved by predicting the mean belief state for all inputs. The ratio MSE/Baseline MSE $= 1 - R^2$ relates the three metrics.

Reporting all three is mildly redundant (any two determine the third) but provides useful sanity checks.

4.5 Missing: Statistical Rigour

The code does not perform train/test splitting of the probing data. All 10^5 data points are used for both fitting and evaluation. With $d = 8$ (or even $d = 32$ for concatenated features) and 10^5 samples, overfitting is unlikely in practice, but a held-out split would strengthen the claims. More importantly, no confidence intervals or significance tests are reported for R^2 differences across layers.

5 Engineering Critique

5.1 Strengths

- **MPI parallelism.** The `run_pca_and_save_results` function distributes model directories across MPI ranks with round-robin GPU assignment. Rank 0 generates belief states once and broadcasts them, avoiding redundant computation. Error handling is robust: each directory is processed in a try/except block with per-rank summaries.
- **Automatic device selection.** The `get_device()` helper correctly prioritises CUDA → MPS → CPU.
- **Skip-if-done logic.** Already-processed directories (containing `pca_data.npz`) are skipped, enabling restartable batch runs.
- **Comprehensive serialisation.** Regressors (pickle), activations/predictions (compressed npz), and metrics (CSV) are all saved, enabling downstream analysis without re-running the pipeline.

5.2 Issues

1. **Unused imports.** `PCA` (line 12), `math` (line 14), `F` (line 7), `transformer_lens` (line 22), and `NNsight` (line 21) are imported but never used in the module's own functions. The `NNsight` import triggers a side effect (registering the `.trace()` context manager on `nn.Module`), which

is required for the `MaskedHeadTransformerModel` branch of `extract_residual_stream`. This implicit dependency should be documented.

2. **Model lookup via `globals()`.** Line 54 resolves the model class by name from `globals()`, which depends on the specific imports at the top of the file (`HookedTransformerModel`, `MaskedHeadTransformerModel`). This is fragile: adding a new model class requires editing the import block. A registry pattern (as in `src/utils.py`'s `PROCESS_REGISTRY`) would be more maintainable.

3. **Hard-coded HMM process.** The entry point (line 518 onward) hard-codes `Mess3Proc()` and the filename `final_model_mess3.pt`. The HMM process should be read from the model's `config.yaml` to support other processes (Z1R, RRXOR, CylinderGraphHMM) without code changes.

4. **Memory scaling.** With `batch_size=100 000` and `seq_length=16`, the belief-state tensor is shape $(10^5, 16, N)$ and the activation tensor is $(L, 10^5, 16, d)$. For a 4-layer model with $d = 8$, this is only $\sim$50 MB. But for the CylinderGraphHMM with $N = 18$, $d = 64$, $L = 4$, the activation tensor alone is $\sim$3.3 GB. The code loads everything into memory simultaneously. For larger models, a streaming or chunked approach would be needed.

5. **Belief states on wrong device.** Line 611 creates `t.tensor(truncated_beliefs, device=device)`, but `truncated_beliefs` is a NumPy array and the belief states are only used by `learn_affine_mapping`, which immediately calls `.cpu().numpy()`. The GPU round-trip is unnecessary.

6. **Redundant computation in `pca_layer_wise_actvs`.** Lines 255–263 recompute the per-layer activation slicing that was already done in the main loop (lines 223–231), solely to save the data. This duplicates the slicing logic and risks inconsistency.

7. **No `run_id` in layer-wise output filenames.** The layer-wise functions save to fixed names (`pca_data.npz`, `pca_metrics.csv`, `regressors.pkl`) without incorporating a run identifier, unlike the concatenated variant which uses `run_id` prefixes. This makes it impossible to store multiple analyses in the same directory.

6 Refactored Implementation

The monolithic `old_src/pca.py` has been decomposed into five modules under `src/`, each with a single responsibility and a corresponding test suite under `tests/`.

6.1 Architecture

`src/linear_probe.py` Core probing logic (pure NumPy/sklearn, no I/O, no Torch). Exposes a `ProbeResult` dataclass and three entry points: `fit_probe_with_split`, `probe_layer_wise`, and `probe_concatenated`. All functions take a `test_fraction` and `seed` argument to produce reproducible train/test splits.

`src/activation_extraction.py` Model loading via `initialize_transformer_from_yaml` (from `src/utils.py`) and residual-stream extraction via TransformerLens's `run_with_cache`. Supports chunked inference (`batch_size` parameter) for large datasets. Auto-detects the best checkpoint file (priority: `best_model.pt` > `latest.pt` > highest-epoch `checkpoint_epoch_*.pt`). The HMM process is read from `config.yaml` via `create_process_from_dict`, eliminating the hard-coded `Mess3Proc` dependency.

`src/visualisation.py` Simplex plotting (matplotlib). Ported from the original code with added `ax` parameter for subplot support and configurable `vertex_labels`.

`src/probe_io.py` Serialisation: `save_probe_results` writes a metrics CSV (with train and test columns), pickled regressors, and optionally compressed NPZ data. All filenames use a consistent `run_id` prefix. `load_probe_results` is the inverse.

`src/run_linear_probe.py` CLI entry point. Supports single-model (`--model-dir`) and batch (`--model-dirs`) modes with optional MPI parallelism. In batch mode, Rank 0 generates shared sequences and belief states, then broadcasts to all ranks—the same pattern as the original code.

6.2 Data Flow

The pipeline proceeds as follows:

1. Read `config.yaml` from the model directory to determine the HMM process and model architecture.
2. Generate observation sequences and compute ground-truth belief states via `HMM.mixed_state_presentation`.
3. Load the trained `HookedTransformer` and extract post-layer residual-stream activations with `extract_residual_stream`.
4. Reshape activations and beliefs via `prepare_probe_data` (last-position or all-position).
5. Fit linear probes via `probe_layer_wise` and/or `probe_concatenated`, each with a proper train/test split.
6. Save results via `save_probe_results`.

6.3 Output Format

Running the CLI produces a `probe_results/` directory inside the model directory (or at a custom `--output-dir`) with files:

- `layerwise_metrics.csv` — one row per layer with columns: `layer`, `train_mse`, `test_mse`, `train_r2`, `test_r2`, `baseline_mse`.
- `concat_metrics.csv` — single row for the concatenated probe.
- `*_regressors.pkl` — pickled `LinearRegression` objects.
- `*_data.npz` — (optional, with `--save-data`) train/test predictions.

7 Usage Guide

7.1 Single Model

```
python src/run_linear_probe.py \
    --model-dir models/cylinderhmm/20250301_run1/
```

7.2 Batch Processing

```
python src/run_linear_probe.py \
    --model-dirs models/cylinderhmm/*/
```

7.3 MPI Parallelism

```
mpirun -n 4 python src/run_linear_probe.py \
    --model-dirs models/cylinderhmm/*/ --use-mpi
```

7.4 Loading Results in a Notebook

```
from probe_io import load_probe_results
results = load_probe_results("models/.../probe_results",
                             run_id="layerwise")
print(results["metrics"])
```

8 Summary of Recommendations

Category	Priority	Recommendation	Status
Conceptual	High	Rename module from `pca.py` to `linear_probe.py` (Section 4.1).	**Resolved**
Conceptual	Medium	Add train/test split to probing evaluation.	**Resolved**
Conceptual	Low	Consider nonlinear probe as comparison baseline.	Open
Engineering	High	Read HMM process from `config.yaml` instead of hard-coding `Mess3Proc`.	**Resolved**
Engineering	Medium	Replace `globals()` model lookup with registry.	**Resolved**
Engineering	Medium	Remove unused imports; drop NNsight dependency.	**Resolved**
Engineering	Low	Eliminate GPU round-trip for belief states.	**Resolved**
Engineering	Low	Unify `run_id` naming convention.	**Resolved**

Table 1: Summary of recommendations with resolution status after refactoring.

References

[1] A. Shai, S. Marzen, L. Teixeira, A. Oldenziel, and P. Riechers, "Transformers represent belief state geometry in their residual stream," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. `arXiv:2405.15943`.

[2] A. Shai, P. Riechers, et al., "Constrained belief updates explain geometric structures in transformer representations," in *International Conference on Machine Learning (ICML)*, 2025. arXiv:2502.01954.

[3] P. Jurgens and J. P. Crutchfield, "Shannon entropy rate of hidden Markov processes," *Journal of Statistical Physics*, vol. 183, 2021.

[4] J. P. Crutchfield and K. Young, "Inferring statistical complexity," *Physical Review Letters*, vol. 63, pp. 105–108, 1989.

[5] G. Alain and Y. Bengio, "Understanding intermediate layers using linear classifier probes," in *ICLR Workshop*, 2017.

[6] N. Aggarwal, S. R. Dalal, and V. Misra, "The Bayesian geometry of transformer attention," arXiv:2512.22471, 2025.